McMaster University
Faculty of Engineering
COMPSCI 4ZP6 - Capstone Project

Software Requirements Specification

ARDEN

AI-Driven Game Narrative Editor

Team 30

Cathy Xu	xu518@mcmaster.ca
Duzhi Wu	wud72@mcmaster.ca
Moein Roghani	roghanim@mcmaster.ca
Xingjian Zheng	zhengx68@mcmaster.ca

Version 0
October 10, 2025

Capstone Instructor: Dr. Mehdi Moradi

Table of Contents

1 Team Information

1.1 Team Member Roles

Team Member	Role	Responsibilities
Cathy Xu	Frontend Lead	React UI development, graph visualization, user interface design
Duzhi Wu	Database & Data Layer Lead	PostgreSQL database integration, Node.js database service development, service layer architecture
Moein Roghani	Backend, AI Integration & Architecture Lead	Backend services, AI pipeline integration, system architecture design, LangGraph implementation, OpenAI API integration, vector embeddings
Xingjian Zheng	Game Developer & Researcher	Unity engine integration, game development research, engine compatibility implementation

1.2 Table of Contributions

Team Member	Sections Contributed
Cathy Xu	5.1 (Priority 0 - UI/Dashboard Requirements), 9.5 (Technology Stack - Frontend)
Duzhi Wu	4.2 (Environment Constraints), 9.1.1 (Two-Repository Architecture - Database Service), 9.5 (Technology Stack - Database)
Moein Roghani	5.1 (AI Pipeline Requirements), 9 (Project Development Plan), 9.1 (Team Meeting and Communication Plan), 9.2 (Team Member Roles), 9.3 (Workflow Plan), 9.4 (Proof of Concept Demonstration Plan), 9.5 (Technology Stack - Backend)
Xingjian Zheng	5.1 (Unity Integration Requirements), 5.4 (Priority 3 - Future Engine Integration), 9.4 (Proof of Concept Demonstration - Unity Integration), 9.5 (Technology Stack - Game Integration)

2 Purpose of the Project

ARDEN (AI-Driven Game Narrative Editor) extends Microsoft's GENEVA research by implementing a native editor architecture for game narrative design. GENEVA, presented at IEEE CoG 2024, demonstrated LLM-based generation of branching narrative graphs using GPT-4 in a two-step process[1]: first generating branching storylines from narrative descriptions and constraints, then producing code to render these narratives in graph format. ARDEN operationalizes this research into a production tool with persistent state management, iterative editing capabilities, and direct game engine integration.

Current game narrative design workflows exhibit five primary inefficiencies that ARDEN addresses:

Tool Fragmentation: Narrative teams use separate applications for story writing (Twine, Ink), world-building (articy), and implementation, requiring manual synchronization between systems. ARDEN provides a unified platform that handles all stages of narrative development.

Manual Graph Maintenance: Existing tools require manual node creation and connection management, limiting iteration speed as narrative complexity increases. ARDEN automates graph construction through AI assistance while maintaining human creative control.

Static AI Generation: Current LLM tools produce complete outputs without supporting incremental modification or context preservation across sessions. ARDEN maintains persistent conversational context, enabling iterative refinement of narrative elements.

Integration Overhead: No direct export pathway from narrative design tools to game engine formats requires custom parsing and integration work. ARDEN provides direct Unity integration through a custom Unity package, available via the Unity Package Manager (UPM) or Unity Asset Store, with automated workspace configuration.

Coherence Management: As story graphs exceed 50+ nodes, maintaining narrative consistency becomes computationally expensive for human designers. ARDEN uses pattern-based validation with regex matching and relationship tracking to automatically detect contradictions and maintain story coherence.

[1] Rao, S., Brockett, C., & Dolan, B. (2024). GENEVA: GENErating and Visualizing branching narratives using LLMs. *IEEE Conference on Games (CoG 2024)*. Microsoft Research. Microsoft Research: GENEVA Project

2.1 System Architecture Overview

ARDEN implements a streamlined three-component architecture:

Unified Frontend-Backend: Monorepo structure with React frontend and Python backend services, deployed as a single unit.

Node.js Database Service: Dedicated service layer for PostgreSQL interactions, providing RESTful endpoints.

PostgreSQL Database: Persistent storage for user data, narrative graphs, and relationship mappings.

3 The Client and Stakeholders

Primary: Game Designers and Developers - Narrative designers and game developers are the primary users of ARDEN. The tool enables them to rapidly create and iterate on narrative structures while maintaining creative control. The direct Unity integration particularly benefits indie developers who lack resources for custom narrative tools.

Secondary: Game Companies and Players - Game development companies benefit from improved efficiency and narrative quality. Players ultimately experience more sophisticated and engaging interactive narratives as a result.

4 Project Constraints

4.1 Partner or Collaborative Application Constraints

- The editor needs to be user-friendly so that users can easily understand how to use it
- The narrative graphs exported from our editor are packaged as a Unity-compatible package, distributable through both Unity Package Manager (UPM) and Unity Asset Store

4.2 Environment Constraints

- The client would have access to website of our editor
- The client would be able to use Unity engine and import our Unity package through either the Unity Package Manager or Asset Store

4.3 Schedule Constraints

- The functioning proof of concept of the editor should be done by November 23, 2025
- A game based on the node graph created by our editor should be down before February 14, 2026
- The editor needs to be completed before April 3, 2026

4.4 Budget Constraints

- The editor would be developed within the budget of 70$
- The editor would upkeep with at most 20$/month

5 Functional Requirements

The following functional requirements are organized by development priority, with Priority 0 representing the minimum viable product necessary for core functionality.

5.1 Priority 0 (Minimum Viable Product)

At this stage, ARDEN delivers foundational components necessary to generate, visualize, and persist interactive story graphs through AI-assisted workflows. These features establish the core link between narrative design and engine-ready implementation.

- Implement a React-based multi-panel dashboard combining AI Assistant panel and Graph Editor view for seamless narrative creation workflow
- Enable node-based story creation supporting essential node types (start, dialogue, choice, end) with intuitive drag-and-drop connections
- Integrate Python backend service using LangGraph framework to process AI generation requests, maintain conversation context, and manage user interactions
- Establish Node.js database service to handle PostgreSQL integration and provide data layer endpoints for frontend and backend communication
- Establish PostgreSQL database for secure storage of user projects, node data, lore information, and complex relationships between characters, events, and locations
- Implement session-based state preservation allowing users to continue working across multiple sessions without losing previous AI context or conversation history

- Support JSON export of complete narrative graphs optimized for Unity integration via a dedicated Unity package, enabling seamless import to game engine workflows through the Unity Package Manager (UPM) or Unity Asset Store
- Include secure authentication and project-based access control to ensure proper user management and data protection
- Provide comprehensive developer documentation covering API routes, database schema, and integration procedures for narrative storage and export functionality

5.2 Priority 1 (Next Feature Set)

This development phase enhances interactivity and consistency by enabling iterative story editing, semantic validation, and extended AI understanding within the collaborative workspace.

- Develop comprehensive Inspector Panel allowing users to modify text content, metadata attributes, and node-specific properties for each narrative element
- Introduce intuitive Lore Bible interface for structured editing and dynamic linking of characters, locations, events, and world-building elements
- Implement real-time bidirectional synchronization between Graph Editor and backend state to reflect AI-driven or manual updates immediately across all interface panels
- Add pattern-based validation and intelligent linting features using regex and context-aware matching to automatically detect narrative contradictions, plot holes, or disconnected story branches
- Incorporate comprehensive version tracking and rollback capabilities, allowing restoration of previous story states for comparison, recovery, and collaborative editing
- Enable incremental file modification through regex-based editing algorithms, minimizing computational overhead and re-generation time for large narrative projects
- Provide robust backend endpoints for project import/export, version retrieval, and collaborative workspace management to improve team workflows and data portability
- Enhance Node.js database service with advanced querying capabilities and optimized connection pooling for improved performance
- Expand backend testing coverage ensuring integrity of AI pipeline operations, version control systems, and narrative graph validation processes

5.3 Priority 2 (Non-critical features)

This development phase focuses on scalability improvements, advanced usability features, and performance optimizations for handling complex or large-scale narrative projects efficiently.

- Enhance graph visualization system with advanced zoom/pan controls, intelligent node color coding, and multi-layer hierarchy display for improved navigation
- Support comprehensive dashboard customization allowing users to resize panels, save preferred layouts, and toggle between different editing modes based on workflow needs
- Add sophisticated character-event-location mapping views for visualizing complex relationships and dependencies across all narrative elements
- Optimize pattern-based search performance using advanced regex techniques to accelerate context retrieval in extensive projects with hundreds of nodes
- Introduce flexible user roles and granular collaboration controls for multi-user editing, shared project management, and team-based development workflows
- Maintain persistent conversational context between editing sessions for long-form narrative design projects requiring extended development cycles
- Implement intelligent performance caching for story graphs exceeding 100+ nodes to ensure smooth rendering, interaction, and real-time updates
- Provide comprehensive analytics visualization including branching density analysis, story depth metrics, and node relationship statistics within the interface

5.4 Priority 3 (Future Application Features)

These advanced features expand ARDEN's applicability to enterprise environments and research settings through automation, deeper AI assistance, and integration with broader narrative development ecosystems.

- Implement intelligent AI-driven suggestion system to recommend alternative dialogue paths, character development arcs, and plot resolution options based on narrative context
- Add automated lore extraction capabilities allowing ARDEN to generate comprehensive character and world data from uploaded scripts, documents, or existing text files
- Extend engine export compatibility to Unreal Engine and Godot through dedicated packages, supporting multi-platform deployment and broader game development ecosystem integration beyond the initial Unity package

- Enable cloud-based team collaboration platform providing real-time editing capabilities, commenting systems, and shared development sessions for distributed teams
- Introduce comprehensive analytics dashboard tracking creative metrics including narrative length analysis, decision complexity measurement, and story coherence trends over time
- Develop real-time Unity synchronization through an enhanced Unity package enabling live graph updates during gameplay testing and iterative development cycles
- Provide extensive API endpoints for external tool integration, supporting plugin development and third-party extension creation for specialized workflows
- Include flexible plugin architecture enabling community-driven narrative templates, custom AI agent configurations, and specialized industry-specific extensions

6 Data and Performance Metrics

6.1 AI Integration

ARDEN integrates with OpenAI's API to provide AI-assisted narrative generation capabilities. The system connects to the OpenAI API for real-time narrative assistance and maintains user-specific context in the PostgreSQL database.

6.2 Performance Metrics

The following system performance metrics will be tracked:

- AI Response Time: Average time for narrative generation requests (target: <2 seconds)
- Graph Rendering Performance: Frame rate for visualizing graphs with 100+ nodes (target: >30 FPS)
- Database Query Response: Average time for save/load operations (target: <1 second)
- Export Processing Time: Duration to generate Unity package-compatible JSON files (target: <5 seconds)

7 Non-functional Requirements

7.1 Data Requirements

The system manages data through the following components:

- PostgreSQL database stores project data including narrative graph structures, node contents, and user metadata
- Lore Bible functionality maintains structured information about characters, locations, and events
- AI pipeline temporarily stores prompt-response pairs for conversation context
- Exported data follows schema standards for Unity package integration, compatible with both UPM and Asset Store distribution methods

7.2 Usability Requirements

ARDEN prioritizes intuitive design for game developers:

- Multi-panel dashboard layout inspired by game development editors
- Graph Editor with drag-and-drop controls and color-coded nodes
- Contextual guidance and AI prompts for narrative generation tasks
- Clear error messages and validation warnings

7.3 Performance Requirements

System performance standards for optimal user experience:

- Graph rendering with <100ms interaction delay for up to 100+ nodes
- Backend response time for AI interactions under 2 seconds
- Database queries complete within 1 second for typical projects
- Unity package-compatible JSON export process completes in under 5 seconds

7.4 Security Requirements

Basic security measures to protect user data:

- Authentication system for user access control
- Secure data transmission and storage

- Project access limited to authorized users

8 Risks and Issues Predicted

8.1 Risks

AI Generation Accuracy: The narrative graphs created by AI assistance may not achieve sufficient quality or accuracy to match user expectations, particularly for complex storylines or specific genre requirements. This risk will be mitigated through iterative testing, user feedback incorporation, and fine-tuning of prompts and context management systems.

Database Integrity Issues: Potential problems include user authentication failures leading to access of incorrect project data, or data corruption resulting in incomplete preservation of narrative graphs including missing nodes or altered connections between story elements. Risk mitigation includes comprehensive automated testing, regular database backups, transaction-based operations, and robust Node.js service layer validation.

Performance and Scalability Concerns: As project data volume grows, the system may experience performance degradation when processing requests, particularly affecting server response times and user experience quality. This will be addressed through performance optimization, caching strategies, and scalable architecture design.

Security Vulnerabilities: System compromises could lead to unauthorized access to user projects and private narrative data, potentially exposing intellectual property or confidential game development information. Mitigation strategies include secure authentication, encrypted data storage, regular security audits, and compliance with industry best practices.

User Interface Complexity: When narrative graphs become heavily populated with numerous nodes and connections, users may experience difficulty selecting, editing, or navigating specific elements, potentially hindering productivity and creative workflow. Solutions include improved visualization tools, filtering options, and intuitive navigation controls.

8.2 Predicted Issues

AI Consistency Challenges: Even with identical input content or keywords, the AI system may generate significantly different narrative graph structures, making it difficult to control output consistency and predict results. This variability could impact user confidence and workflow reliability, requiring development of consistency

mechanisms and user education about AI capabilities and limitations.

Integration Complexity: Unity package integration may encounter compatibility issues across different Unity versions, requiring ongoing maintenance and updates to ensure seamless workflow integration for diverse development environments. The Unity package must maintain compatibility with both UPM and Asset Store distribution methods while adhering to Unity's packaging standards.

Resource Management: OpenAI API costs may exceed budget constraints during periods of high usage or complex narrative generation, necessitating usage monitoring, optimization strategies, and potential alternative AI model integration.

Learning Curve: Users transitioning from traditional narrative tools may require significant time investment to adapt to AI-assisted workflows, potentially affecting initial adoption rates and requiring comprehensive training materials and support documentation.

9 Project Development Plan

9.1 Team Meeting and Communication Plan

9.1.1 Communication Platforms

- **Primary Communication**: Group chats for real-time team communication and updates
- **Documentation Sharing**: Google Drive and Overleaf for collaborative document editing
- **Meeting Platform**: Microsoft Teams for virtual meetings
- **Project Management**: Jira for Kanban-style task tracking

9.1.2 Meeting Schedule

- **Backlog Assessment Meetings**: Every Wednesday 4:30-5:30 PM (weekly)
  - Review current backlog status and story progression
  - Brief updates on current story progress (standup)
  - Identify blocked stories and dependency resolution
  - Assess project velocity and timeline adjustments
  - Prioritize upcoming stories based on current development state

9.1.3 Documentation Management

- All meeting notes recorded in shared Google Doc with action items tracked
- Backlog assessment documentation maintained in Jira project boards
- Technical decisions and architecture changes documented in repository README files

9.2 Team Member Roles

9.2.1 T-Shaped Team Composition

The team employs a T-shaped role concept where each member combines deep expertise in one domain with broader knowledge across multiple areas. This approach enables cross-functionality within the team given our limited resources, encouraging knowledge sharing while maintaining specialized expertise in each domain.

9.2.2 Team Leadership Structure

- **Project Manager**: Moein Roghani
  - Overall project timeline management and stakeholder/core team communication
  - Backlog prioritization coordination and resource allocation

9.2.3 Functional and Non-Functional Requirements Assignment

The team member roles and responsibilities follow the assignments detailed in:
Section 1.1: Team Member Roles
Section 1.2: Table of Contributions

The assignment of tasks is organized according to priority levels in:
Section 6: Functional Requirements
Section 7: Non-functional Requirements

9.3 Workflow Plan

9.3.1 Repository Architecture and Deployment Strategy

Two-Repository Architecture

- **Database Service Repository** (Separate Repository)
  - **Technology**: Node.js application serving as exclusive database interface layer
  - **Purpose**: Handles all PostgreSQL interactions, query optimization, and data

layer operations
  - **API**: Provides RESTful endpoints for the main ARDEN application
- **ARDEN Main Application Repository** (Monorepo)
  - **Frontend**: React-based static frontend application
  - **Backend**: Python application server with AI pipeline integration
  - **Architecture**: Unified deployment where static React build is served by Python backend
  - **Structure**: Single deployment unit eliminating frontend/backend coordination complexity
- **PostgreSQL Database**
  - **Access**: Exclusively through Node.js database service to ensure data consistency
  - **Storage**: User authentication, project data, narrative graphs, and relationship mappings
  - **Optimization**: Dedicated service layer provides optimized performance and connection pooling

Branching Strategy

- **Main Branch**: Stable code with version tagging
- **Feature Branches**: Individual feature development

Code Review Process

1. All code changes require pull request creation with standard PR template
2. Code review and approval required before merging to main branch
3. Versioning templates will be used for release management
4. Issue management handled through Jira rather than GitHub Issues

9.3.2 Development Methodology

Agile-Influenced Framework ARDEN employs an agile-influenced framework based on agile principles for project management of the software's life cycle. This hybrid approach balances flexibility with static goals and a structured roadmap, allowing the team to build iteratively while maintaining focus on established milestones and requirements.

Milestone-Based Development

- Development organized around key milestones with clear objectives
- Structured release plan based on known requirements
- Backlog refinement to prioritize work items according to milestone goals
- No story point estimation or sprint-based development

Development Cadence

- **Weekly Backlog Refinement**: Wednesday meetings serve as the primary cadence for project coordination
- **Combined Stand-up**: Status updates integrated into weekly backlog meetings
- **Continuous Flow**: Work items move through development stages without time-boxed iterations

9.3.3 Data Storage and Infrastructure

Development Data Management

- **Local Development**: Local database instances for development and testing
- **Testing Environment**: Shared PostgreSQL instance for integration testing

AI Resources

- **Development**: OpenAI API for development and testing
- **Model Selection**: OpenAI model to be determined based on project requirements
- **Conversation Management**: Session-based context management for AI interactions

9.4 Proof of Concept Demonstration Plan

9.4.1 Demonstration Objectives

The proof of concept will showcase ARDEN's initial implementation focusing on the core research and AI-assisted narrative design capabilities. This demonstration represents the first milestone in the project development cycle.

9.4.2 Core Features to Demonstrate

Web-Based Platform

- Functional React-based user interface with basic component structure
- Initial implementation of the narrative design workspace
- Core UI components for user interaction with the AI system

AI-Assisted Narrative Design

- Integration with OpenAI API for narrative generation
- Implementation of the agentic system design for narrative assistance
- Demonstration of the research-based approach to narrative design

Unity Integration

- Prototype Unity package demonstrating narrative graph import functionality
- Basic implementation of the Unity-compatible JSON schema
- Sample Unity project showcasing integration with exported narrative graphs
- Package structure following Unity Package Manager standards
- Demonstration of both UPM and Asset Store distribution methods

Basic System Architecture

- Frontend-backend communication pathways
- Initial database structure for storing narrative elements
- Core system components and their interactions

9.5 Technology Stack

Component	Technologies
Frontend	React 18.2+, JavaScript
Backend	Python 3.11+, LangGraph, OpenAI Python SDK
Database	PostgreSQL
Database Service	Node.js, Express.js
Game Integration	Unity Package (distributed via Unity Package Manager and/or Unity Asset Store)

9.6 Project Scheduling

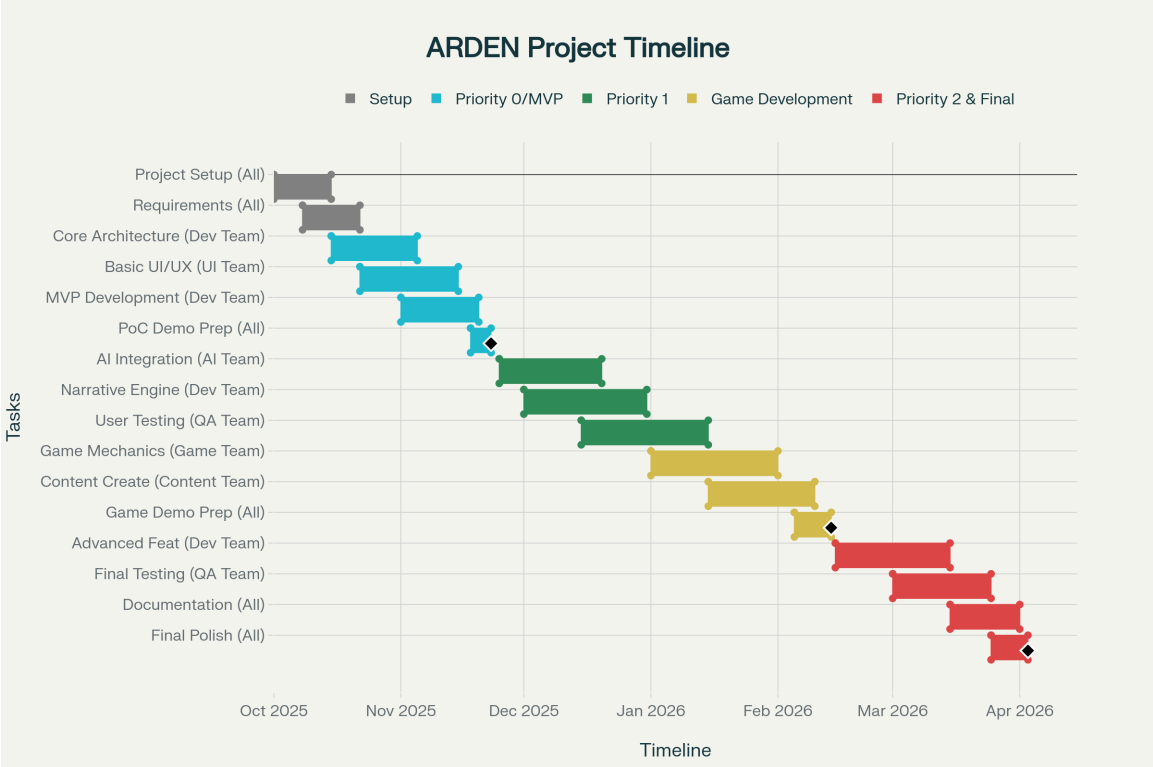

Figure 1: Gantt chart showing project timeline, milestones, and task dependencies.

10 References

1. Rao, S., Brockett, C., & Dolan, B. (2024). GENEVA: GENErating and Visualizing branching narratives using LLMs. *IEEE Conference on Games (CoG 2024)*. Microsoft Research. Available at: Microsoft Research: GENEVA Project
2. Microsoft Research. (2024). Player-Driven Emergence in LLM-Driven Game Narrative. *IEEE Conference on Games (CoG 2024)*. Available at: Microsoft Research Website
3. Inworld AI & Xbox Collaboration. (2024). AI in Game Development: Transformative Technology for Creators. Microsoft Research. Available at: Microsoft Research Website

11 Abbreviations and Definitions

11.1 Abbreviations

Term	Definition
AI	Artificial Intelligence
API	Application Programming Interface
ARDEN	AI-Driven Game Narrative Editor. Named after Shakespeare's Forest of Arden from "As You Like It," symbolizing a creative sanctuary where transformation, imagination, and artistic freedom flourish
CLI	Command Line Interface
GENEVA	Microsoft's GENEVA research project: "GENErating and Visualizing branching narratives using LLMs" presented at IEEE CoG 2024
JSON	JavaScript Object Notation
LLM	Large Language Model
MVP	Minimum Viable Product
NPC	Non-Player Character
PostgreSQL	Open-source relational database management system
React	JavaScript library for building user interfaces
REST	Representational State Transfer
SRS	Software Requirements Specification
SVG	Scalable Vector Graphics
UI	User Interface
UPM	Unity Package Manager - System for installing and managing Unity packages
UX	User Experience

11.2 Definitions

Term	Definition
Narrative Graph	A directed graph structure representing story flow with nodes for dialogue, choices, and story events
Lore Bible	A structured database containing characters, locations, events, and their relationships within a game world

Node Types	Categories of story elements: start, dialogue, choice, action, merge, climax, end
Session Context	Persistent AI conversation state that maintains narrative understanding across editing sessions
Pattern-Based Indexing	Search system that uses regex patterns and context-aware matching to find and manage narrative graph elements